Detecção de Bad Smells e Refatoração Segura

Disciplina: Teste de Software

Trabalho: Bad Smells com ESLint + SonarJS

Aluno(a): Gabriela Alvarenga Cardoso

Matrícula: 844469

Projeto: bad-smells-js-refactoring (ReportGenerator)

1. Análise de Smells (manual)

Leitura manual do método `generateReport` de `src/ReportGenerator.js` (um único método de ~55 linhas com todo o comportamento do relatório).

1.1 Método Longo / Complexidade Ciclomática Alta

Um único método monta cabeçalho, corpo, rodapé, aplica regras de permissão e regras de prioridade, com loop e quatro níveis de *if/else* aninhados. Para testar um ramo é preciso percorrer toda a cadeia; qualquer alteração (ex.: novo perfil) exige entender o método inteiro, e o risco de regressão é alto.

1.2 Duplicação de Lógica

A linha de CSV (`` `${item.id},${item.name},...` ``) e a de HTML aparecem duas vezes cada (ramo ADMIN e ramo USER), assim como `total += item.value`, que se repete 4 vezes. Corrigir um formato exige lembrar de editar todas as cópias; a duplicação foi, de fato, a origem de um bug (ver 1.4).

1.3 Números Mágicos e Condicionais por tipo (Switch Statements)

Os valores `1000` e `500` aparecem sem nome, escondendo regras de negócio. Além disso, `reportType === 'CSV'/'HTML'` e `user.role` são testados em vários pontos; incluir um novo formato (ex.: JSON) obrigaria alterar cabeçalho, corpo e rodapé (Shotgun Surgery).

1.4 Efeito colateral no código original (observação extra)

O ramo ADMIN/HTML gera `<tr ${style}>`; quando o item não é prioritário o resultado é `<tr >` (espaço sobrando). O teste *"relatório HTML completo para Admin"* espera `<tr>`, e por isso **falha já no código original** (ver seção 3). Também há mutação do parâmetro `items` (`item.priority = true`).

2. Relatório da Ferramenta (ESLint + SonarJS)

Configuração: `eslint:recommended` + `plugin:sonarjs/recommended`, `cognitive-complexity` limite 5, `no-identical-functions e no-duplicated-branches` (arquivo `.eslintrc.json` já presente no repositório). Saída de `npx eslint src/` antes da refatoração:

```
src/ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 5 allowed  sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one                                          sonarjs/no-collapsible-if

2 problems (2 errors, 0 warnings)
```

Na leitura manual eu percebi que o método era "grande e cheio de ifs", mas a ferramenta **quantificou** o problema: complexidade cognitiva 27 contra um limite de 5, pois a métrica pondera o aninhamento (cada *if* dentro de outro pesa mais), algo difícil de estimar a olho. Ela também apontou o *if* aninhado colapsável (linha 43, `else if (user.role === 'USER') { if (item.value <= 500) ...`), que passou despercebido na análise manual. Por outro lado, a ferramenta não vê a duplicação de strings nem os números mágicos, que só a análise manual encontrou: os dois métodos se complementam.

3. Processo de Refatoração

Smell escolhido: o **Método Longo com condicionais aninhadas e duplicadas** (`generateReport`, complexidade 27). Trabalhei em `src/ReportGenerator.refactored.js`, com uma cópia do teste apontando para ele (`tests/ReportGenerator.refactored.test.js`, apenas o *import* alterado). Os testes originais não foram editados.

Antes (corpo do método, ramos por perfil x formato)

```
    for (const item of items) {
      if (user.role === 'ADMIN') {
        // Admins veem todos os itens
        if (item.value > 1000) {
          // Lógica bônus para admins
          item.priority = true;
        }

        if (reportType === 'CSV') {
          report += `${item.id},${item.name},${item.value},${user.name}\n`;
          total += item.value;
        } else if (reportType === 'HTML') {
          const style = item.priority ? 'style="font-weight:bold;"' : '';
          report += `<tr ${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
          total += item.value;
        }
      } else if (user.role === 'USER') {
        // Users comuns só veem itens de valor baixo
        if (item.value <= 500) {
          if (reportType === 'CSV') {
            report += `${item.id},${item.name},${item.value},${user.name}\n`;
            total += item.value;
          } else if (reportType === 'HTML') {
            report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
            total += item.value;
          }
        }
      }
    }
```

Depois (método + estruturas auxiliares)

```
function selectVisibleItems(policy, items) {
  if (!policy) {
    return [];
  }
  return items.filter((item) => {
    policy.prepare(item);
    return policy.isVisible(item);
  });
}

export class ReportGenerator {
  constructor(database) {
    this.db = database;
  }

  /**
   * Gera um relatório de itens baseado no tipo e no usuário.
   * - Admins veem tudo (itens acima de 1000 ganham destaque).
   * - Users comuns só veem itens com valor <= 500.
   */
  generateReport(reportType, user, items) {
    const formatter = lookup(FORMATTERS, reportType);
    if (!formatter) {
      return '';
    }

    const policy = lookup(ROLE_POLICIES, user.role);
    const visibleItems = selectVisibleItems(policy, items);

    const body = visibleItems
      .map((item) => formatter.row(item, user, policy.isHighlighted(item)))
      .join('');
    const total = visibleItems.reduce((sum, item) => sum + item.value, 0);

    return (formatter.header(user) + body + formatter.footer(total)).trim();
  }
}

const csvFormatter  = { header, row, footer };   // formato CSV
const htmlFormatter = { header, row, footer };   // formato HTML
const FORMATTERS    = { CSV: csvFormatter, HTML: htmlFormatter };
const ROLE_POLICIES = { ADMIN: adminPolicy, USER: userPolicy };  // isVisible / prepare / isHighlighted
```

Técnicas aplicadas: *Extract Method* / *Extract Function* (`selectVisibleItems`); *Replace Conditional with Polymorphism* (tabelas de objetos com a mesma interface: `FORMATTERS` para CSV/HTML e `ROLE_POLICIES` para ADMIN/USER, substituindo os if/else em `reportType` e `user.role`); *Replace Magic Number with Symbolic Constant* (`ADMIN_PRIORITY_THRESHOLD`, `USER_MAX_VISIBLE_VALUE`); e eliminação de duplicação (cada linha de relatório e o somatório existem uma única vez; o total vira um `reduce`). Adicionar um formato ou perfil agora significa registrar um novo objeto, sem tocar no método principal.

Resultado do ESLint (refatorado): `npx eslint src/ReportGenerator.refactored.js` não reporta nenhum problema (2 erros → 0, complexidade 27 → abaixo de 5).

Resultado dos testes: a cópia de teste do arquivo refatorado passa **5/5**. Na suíte original (arquivo antigo), 4/5 passam; o teste de HTML para Admin falha por causa do `<tr >` do código legado (item 1.4). A versão refatorada gera `<tr>` corretamente, portanto a única mudança de comportamento é a correção desse bug. Total da execução: 9 de 10 testes passando (o único que falha é o do código original, intocado).

4. Conclusão

A suíte de testes foi a rede de segurança que permitiu reorganizar toda a estrutura do método sem medo: a cada passo, bastava rodar os testes para saber se o comportamento observável continuava igual. Ela também revelou um bug latente (o espaço em `<tr >`) que a leitura do código sozinha não mostrava. A redução dos smells (complexidade 27 → abaixo de 5, nenhuma duplicação, constantes nomeadas) deixa o código mais legível, mais fácil de testar em partes isoladas e mais barato de evoluir, pois novas regras viram novos objetos em vez de novos ramos no meio de um método gigante. A análise estática e a análise manual se complementaram: a ferramenta mediu a complexidade; a leitura humana encontrou a duplicação e os números mágicos.